

AWS Retail ETL Pipeline

Project Report

Project Title

AWS Retail ETL Pipeline using Serverless AWS Services

Objective

The objective of this project is to automate the processing of retail sales data using AWS serverless services. The pipeline validates incoming files, performs ETL operations, loads transformed data into Amazon Redshift, and archives processed files.

Technologies Used

- Python
 - Amazon S3
 - AWS Lambda
 - AWS Glue
 - Amazon Redshift
 - AWS IAM
 - Amazon CloudWatch
-

Project Architecture

The project follows a serverless ETL architecture where data is uploaded to Amazon S3 and processed automatically through AWS Lambda and AWS Glue before being loaded into Amazon Redshift.

Flow:

S3 Upload → File Validation Lambda → ETL Orchestrator Lambda → AWS Glue Jobs → Amazon Redshift → Archive Lambda

Project Components

1. File Validation Lambda

File: `file_validation_lambda.py`

Responsibilities:

- Validates CSV file extension.
 - Checks file size.
 - Verifies mandatory columns.
 - Moves invalid files to the rejected folder.
 - Generates execution logs.
 - Invokes the ETL Orchestrator Lambda after successful validation.
-

2. ETL Orchestrator Lambda

File: `ETL_orchestrator_lambda.py`

Responsibilities:

- Starts AWS Glue Job 1.
- Waits for job completion.
- Starts AWS Glue Job 2.
- Executes SQL scripts in Amazon Redshift.
- Invokes the Archive Lambda after successful execution.

Note: SQL scripts are not included in this public repository.

3. Archive Lambda

File: `archive_lambda.py`

Responsibilities:

- Archives curated datasets.
 - Moves processed raw files to the archive folder.
 - Deletes temporary processed files.
 - Preserves the required S3 folder structure.
-

S3 Folder Structure

```
raw/  
├── sales/  
└── archive/  
  
processed/
```

```
curated/

archive/

rejected/

logs/
└─ execution-logs/
```

ETL Workflow

1. Upload retail sales CSV file to Amazon S3.
 2. Validate the uploaded file.
 3. Execute AWS Glue ETL jobs.
 4. Load transformed data into Amazon Redshift.
 5. Archive processed files.
 6. Store execution logs for monitoring.
-

Error Handling

- Invalid file extension detection
 - Empty file validation
 - Mandatory column validation
 - Execution logging
 - Failed job monitoring
-

Future Enhancements

- Replace Lambda orchestration with **AWS Step Functions**.
 - Use **Apache Airflow (Amazon MWAA)** for workflow scheduling and monitoring.
 - Add automated email notifications using Amazon SNS.
 - Implement CI/CD deployment using GitHub Actions or AWS CodePipeline.
-

Conclusion

This project demonstrates a scalable and automated ETL pipeline using AWS serverless services. By combining Amazon S3, AWS Lambda, AWS Glue, and Amazon Redshift, the solution efficiently validates, transforms, loads, and archives retail sales data while minimizing operational overhead.